ReCircle: Project Documentation Pack

Smart Recycling Assistant for Code Clash 2.0 (Track 06) · Computer Vision × Digital Twin × Generative AI

Prepared 9 October 2026 for the event on 15–16 October 2026.

In this pack: 1. PRD · 2. TRD · 3. Security & Access · 4. Frontend Spec · 5. Feature Ticket List · 6. Folder & File Structure

Working assumptions

- ReCircle is a working title.
- Team of 2 to 4. Tickets are tagged by role (ML, BE, FE), so one person can hold several roles.
- The stack follows skills already visible in your repos: Python/FastAPI, React/TypeScript, PyTorch.
- Code Clash requires all code to be written during the event, so this pack is planning only. Confirm with the organizers whether reusing your own boilerplate is allowed. Public datasets and pretrained weights count as open-source tools, which the FAQ allows.

1. Product Requirements Document (PRD)

1.1 Summary

ReCircle is a web app that helps people sort waste correctly and shows what that behavior does to a whole recycling system. A user points a camera at an item. Computer Vision names the material and says whether it is recyclable, reusable, biodegradable, hazardous, or residual. Every scan feeds a Digital Twin of a campus waste flow (source, bin, collection, facility). A Generative AI advisor reads the twin's computed metrics and gives personalized recommendations that push the system toward a circular economy.

1.2 Problem

- Contamination (the wrong item in the wrong bin) cuts recovery yield for a whole batch, not just one item.
- People are unsure about edge cases: greasy pizza boxes, batteries, mixed-material packaging, clothes.
- Facility managers see totals after the fact and cannot test changes (more bins, a new pickup schedule, an awareness campaign) before paying for them.
- Reuse and repair options are rarely offered at the moment of disposal.

1.3 Goals

1. Classify an item in under 3 seconds from a phone or webcam.
2. Show how individual scans change system-level metrics in real time.
3. Let a manager run what-if scenarios and see predicted impact.
4. Generate recommendations grounded in the twin's numbers, never invented numbers.
5. Ship a deployed, demoable product before the 4:30 PM deadline on 16 October.

1.4 Non-goals

- Real IoT sensor integration, robotics, or physical bin hardware.
- Production-grade multi-tenant accounts, billing, or compliance certification.
- Region-perfect disposal law coverage. We ship one configurable rule set for a campus scenario.
- Multi-item detection in one frame (stretch only).

1.5 Users

Persona	Needs	Main screens
Resident / Student (Asha)	A quick answer on where an item goes, simple tips, her own impact	Scan, History, Impact
Facility Manager (Ravi)	See contamination hotspots, test what-if changes, get a report	Twin, What-If, Facility, Reports
Judge / Visitor	Understand the idea in 60 seconds, try a scan with no signup	Demo landing, Scan, Twin

Persona names are fictional.

1.6 User stories

- As a resident, I take a photo of an item and learn its category, the right bin, and how to prepare it.
- As a resident, when the model is unsure, I get a clear 'not sure' message with a safe default instead of a wrong confident answer.
- As a resident, I see how my scans reduced landfill waste and CO2e.
- As a manager, I see the flow from sources to bins to facilities with live counts.
- As a manager, I move sliders (compliance, pickup frequency, bins) and compare a scenario against the baseline.

- As a manager, I click Generate advice and get ranked actions with expected impact and effort.
- As a visitor, I use demo mode without creating an account.

1.7 Functional requirements

ID	Requirement	Priority
FR-01	Capture a photo by camera or file upload	P0
FR-02	Classify into recyclable / reusable / biodegradable / hazardous / residual, with material subtype, confidence, and top-3	P0
FR-03	Show disposal guidance per class (bin, prep steps, reason) from a rule file	P0
FR-04	Low-confidence handling: clear message plus fallback classifier; uncertainty that leans hazardous gets a cautious message	P1
FR-05	Each scan writes an event that updates the twin	P0
FR-06	Twin view: graph of sources, bins, collection, and facilities with live counts and flow weights	P0
FR-07	What-if simulator with sliders and baseline comparison	P0
FR-08	Impact metrics: diversion rate, contamination rate, CO2e avoided, estimated pickup cost	P0
FR-09	AI advisor: structured, personalized recommendations from scan history and twin state	P0
FR-10	Advisor chat for reuse, repair, and upcycling questions	P1
FR-11	Exportable sustainability report	P1
FR-12	Auth with roles (Resident, Manager, Admin) and a guest demo mode	P1
FR-13	Facility dashboard: bin fill levels and contamination hotspots	P1
FR-14	Demo mode: seeded data and a traffic generator that keeps the twin moving	P0

ID	Requirement	Priority
FR-15	Reuse and donation suggestions with local-partner placeholders	P2
FR-16	Leaderboard and streaks	P2

1.8 Non-functional requirements

- **Performance:** scan p95 under 3 s end to end, classifier under 500 ms on CPU, twin update under 1 s, advisor under 6 s.
- **Reliability:** every external call (LLM, fallback model) has a timeout and a fallback. The demo path must work with the LLM offline using cached advice.
- **Usability:** mobile-first scan flow, large touch targets, readable in daylight.
- **Accessibility:** WCAG AA contrast, keyboard operable, screen-reader labels on results.
- **Privacy:** images are processed in memory and not stored by default (see Security & Access).

1.9 Success metrics

Metric	Target
Top-1 accuracy on 5 grouped classes, held-out test set	⩾ 85%
Recall on the hazardous class	⩾ 90% (safety-critical; accept lower precision)
Scan latency p95	< 3 s
Demo completes without a manual restart	3 of 3 rehearsals
Advisor figures match twin figures	100% (validated in code)

1.10 Judging alignment

Criterion	Weight	How ReCircle scores
Technical implementation	30%	Trained model with an eval report, a real simulation engine, a deployed full stack

Criterion	Weight	How ReCircle scores
Innovation & creativity	25%	Closed loop: scans drive the twin, the twin grounds the LLM, and a manager can test changes before spending
Practicality & impact	20%	Campus pilot framing, measurable KPIs (diversion, contamination, CO2e), cost view
Presentation & demo	15%	A 3-minute story: live scan, twin update, advice
Teamwork	10%	Clear role split (ML, BE, FE) shown in the deck

1.11 Demo script (3 minutes)

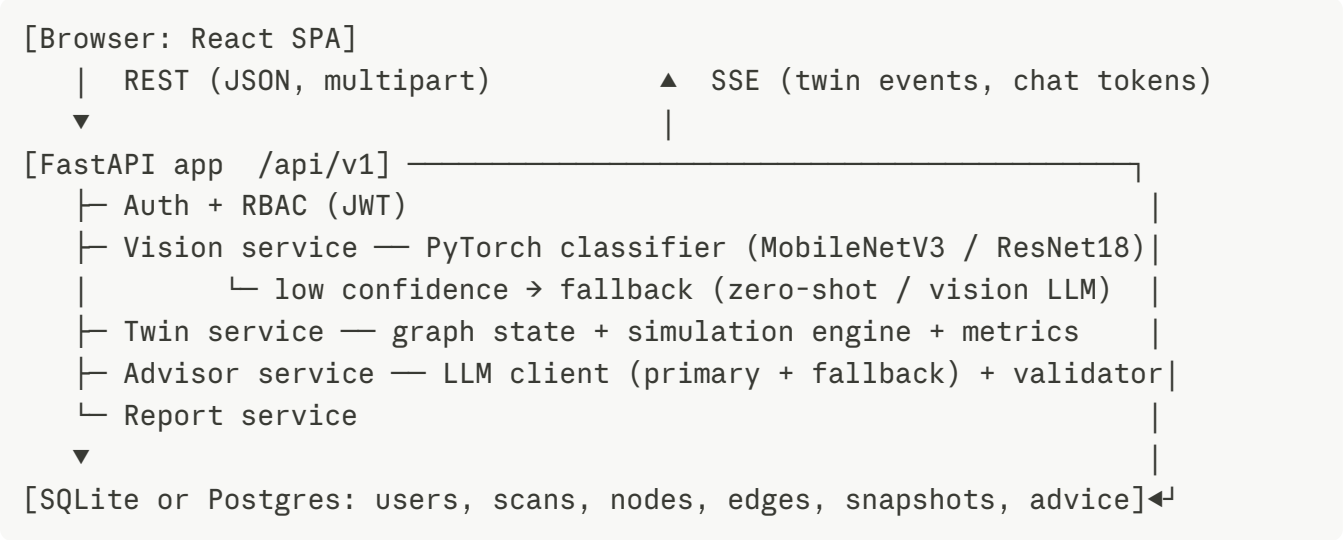
1. **0:00** State the problem in one sentence with one sourced statistic.
2. **0:20** Scan a plastic bottle, a battery, and a banana peel live. Show class, bin, and tip.
3. **1:00** Switch to the twin. The three scans appear as flow and the contamination metric updates.
4. **1:40** What-if: raise compliance from 60% to 80%. Show diversion and CO2e deltas against baseline.
5. **2:20** Click Generate advice: ranked actions with effort and impact.
6. **2:45** Close with the architecture slide, team roles, and next steps.

1.12 Risks and mitigations

Risk	Mitigation
Dataset class imbalance or domain shift (clean dataset photos vs real phone photos)	Augmentation, test on your own phone photos, confidence threshold with fallback
The twin looks fake	Show assumptions on screen, label simulated vs real events, expose every parameter
LLM outage or slow API	Failover provider, response cache, pre-generated advice for the demo scenario
Venue Wi-Fi is unreliable	Local docker compose fallback and a recorded backup video
Scope creep	P0 freeze on Day 2 at 1:00 PM

2. Technical Requirements Document (TRD)

2.1 Architecture



2.2 Tech stack

Layer	Choice	Why
Frontend	React 18, TypeScript, Vite, Tailwind, TanStack Query, Zustand, React Flow, Recharts	Fast to build, strong graph and chart support
Backend	Python 3.11, FastAPI, Pydantic v2, SQLAlchemy	Same language as the ML code, auto-generated OpenAPI docs
ML	PyTorch, torchvision, MobileNetV3-Small or ResNet18 with transfer learning	Small and CPU-friendly at inference
Database	SQLite locally, Postgres if the host provides it	Zero setup
Realtime	Server-Sent Events	One-way twin updates, simpler than WebSockets
LLM	Provider-agnostic client (OpenAI-compatible endpoints or the Anthropic API), primary plus fallback	Avoids a single point of failure
Hosting	Frontend on Vercel or Netlify; backend on Render, Railway, or Hugging Face Spaces (Docker)	Free tiers, quick deploy

Layer	Choice	Why
Tooling	Docker Compose, Makefile, pytest, ESLint, Ruff	Reproducible local runs

2.3 Computer vision module

Classes. Map a public 12-class garbage dataset (for example the Kaggle Garbage Classification set) into five output classes. Check the dataset's licence and actual class list when you download it.

Source label	Output class	Subtype
cardboard, paper	recyclable	paper / cardboard
plastic	recyclable	plastic
metal	recyclable	metal
white / brown / green glass	recyclable	glass
clothes, shoes	reusable	textile / footwear
biological	biodegradable	organic
battery	hazardous	battery
trash	residual	mixed / non-recyclable

- **Model:** fine-tune a pretrained MobileNetV3-Small or ResNet18 (ImageNet weights) at 224 px for 5 to 8 epochs. Augment with flips, rotation, color jitter, and random crops. Use a class-weighted loss to protect hazardous recall.
- **Inference:** softmax, top-3, temperature-scaled confidence. Start with a 0.60 confidence threshold and tune it on the validation set.
- **Fallback:** below the threshold, call a zero-shot or vision-LLM check. If still unsure, return `unsure`. When the hazardous probability is above 0.20, the guidance is: do not put it in a general bin, hand it to facility staff.
- **Evaluation:** confusion matrix, per-class precision and recall, plus a small set of your own phone photos as a realism check. Put the numbers in the deck.
- **Serving:** load the model once at startup, run on CPU in torch inference mode, and resize and normalize server-side.

2.4 Digital Twin

Graph

- Nodes: Source (hostel, canteen, library, department blocks), Bin (one per stream), Collection (route), Facility (recycling, compost, hazardous handling, reuse and donation, landfill).
- Edges: directed flow with a weight (items per tick) and a routing probability.
- State per node: counts by class, fill level (bins), contamination rate (bins), throughput (facilities).

Simulation (discrete ticks in a plain Python loop; one tick is one simulated day)

1. Generate items per source from a seeded, editable waste-mix distribution.
2. Each item has a true class. The resident puts it in the right bin with probability equal to the compliance parameter, otherwise in a wrong bin.
3. Wrong placements raise bin contamination. Contamination above a threshold downgrades that bin's batch (yield loss).
4. Collection empties bins on a schedule. Facilities apply yield (for example recycling 0.85 for a clean batch, 0.55 for a contaminated one).
5. Metrics update.

A real scan adds one item with its predicted class and, if the user confirms the bin, the actual routing.

Metrics

- Diversion rate = items recovered ÷ total items.
- Contamination rate = misrouted items ÷ items in recyclable and organic bins.
- CO2e avoided = Σ mass × (virgin factor − recycled factor). Mass comes from an average-item-weight table and all factors live in `emission_factors.json`. The values you commit first are placeholders. Replace them with a cited source (for example EPA WARM) before the demo and cite it on the slide.
- Pickup cost = trips × cost per trip.

Scenarios. Parameters: `compliance`, `awareness_boost`, `pickup_frequency`, `bins_per_stream`, `facility_yield_override`. The engine runs baseline and scenario on the same random seed and returns deltas.

Principle: the twin computes, the LLM explains.

2.5 Generative AI advisor

- **Input:** a JSON summary built by code: top metrics, top 3 contamination causes (class pairs), hotspot nodes, scenario deltas, the user's scan stats, and relevant snippets from a local disposal knowledge base.

- **Output contract** (validated with Pydantic): `summary`, `issues[]`, `actions[{title, why, expected_impact, effort, owner}]`, `reuse_ideas[]`, `disclaimer`.
- **Rule:** the model may not introduce numbers. A validator checks that every figure in the output appears in the input. On failure, regenerate once, then fall back to a template.
- **Client:** provider-agnostic, primary plus fallback, 20 s timeout, one retry, response cache keyed by the input hash.
- **Chat:** same client with a system prompt limited to waste, reuse, and circular-economy topics. Tokens stream over SSE.

2.6 Data model

Table	Key columns
users	id, email, password_hash, role, created_at
scans	id, user_id, created_at, predicted_class, subtype, confidence, source_node_id, confirmed_bin, image_ref (nullable)
twin_nodes	id, type, name, config_json
twin_edges	id, from_id, to_id, weight, route_prob
twin_snapshots	id, tick, metrics_json, created_at
scenarios	id, user_id, params_json, result_json, created_at
advice_log	id, user_id, input_hash, output_json, provider, latency_ms

2.7 API (base path `/api/v1`)

Method	Path	Purpose	Minimum role
POST	/auth/login, /auth/refresh	Sign in, rotate token	public
POST	/scan	Multipart image plus optional `source_node_id`; returns classification and guidance	Guest
GET	/scans	Scan history	Resident
GET	/twin	Graph plus current metrics	Guest
GET	/twin/stream	SSE twin events	Guest

Method	Path	Purpose	Minimum role
POST	/simulate	Params in; baseline, scenario, and deltas out	Guest (rate-limited)
POST	/advice	Structured advice (guests get a cached sample)	Resident
POST	/chat	Streaming advisor chat	Resident
POST	/reports	Generate report	Manager
GET	/health, /version	Health checks	public
POST	/admin/demo/reset, /admin/demo/traffic	Reset or drive demo data	Admin

Error format: `{ error: { code, message, request_id } }`. OpenAPI docs at `/docs` are disabled in production or placed behind admin.

Example scan response:

```
scan_id: 8f3c1a
predicted_class: hazardous
subtype: battery
confidence: 0.93
top3: [battery 0.93, metal 0.04, trash 0.02]
guidance:
  bin: Hazardous collection point
  steps: [Do not crush, Tape terminals if loose, Hand to facility staff]
  why: Lithium cells can ignite in compactors
twin_effect: { node: Hostel A, contamination_delta: 0.0 }
latency_ms: 410
```

2.8 Realtime

The SSE channel emits `scan.created`, `twin.tick`, and `metrics.updated`. The client reconnects with backoff and falls back to polling every 3 seconds.

2.9 Deployment and environments

- **Local:** `docker compose up` starts the API and web app.
- **Demo:** deployed frontend and backend. Ping `/health` 10 minutes before presenting to avoid cold starts. The model loads at startup.
- **Backup:** the same compose stack on a laptop, plus a recorded backup video.

- **Config:** all secrets and provider settings come from environment variables (listed in section 6).

2.10 Testing

- **ML:** eval script on a fixed test split, plus a smoke set of your own phone photos.
- **Backend:** pytest for twin determinism (same seed, same result), API contract tests, and the advisor number-validator.
- **Frontend:** component smoke tests and a manual script for the demo path.
- **Release gate:** `scripts/smoke_test.sh` must exit 0 against the deployed URL.

3. Security & Access

Scope: a two-day hackathon build that could credibly grow into a campus pilot. Controls are chosen to be cheap to implement now and honest about what is not covered.

3.1 Roles and access matrix

Capability	Guest (demo)	Resident	Manager	Admin
Scan an item	Yes (rate-limited)	Yes	Yes	Yes
View twin graph and metrics	Read-only, demo data	Yes	Yes	Yes
Run what-if simulation	Limited (low quota)	Yes	Yes	Yes
View own scan history	No	Yes	Yes	Yes
Generate AI advice	Cached sample only	Yes	Yes	Yes
Advisor chat	No	Yes	Yes	Yes
Edit twin config (nodes, schedules, yields)	No	No	Yes	Yes
Generate and export reports	No	Own summary	Yes	Yes
Manage users, reset or drive demo data	No	No	No	Yes
View audit logs	No	No	No	Yes

Rule: deny by default. Every route declares its minimum role, and ownership is checked on every record fetch (a resident can only read their own scans).

3.2 Authentication and sessions

- Email and password, hashed with argon2 (or bcrypt with a work factor of 12 or more). Never store or log plaintext passwords.
- Short-lived JWT access token (15 minutes) plus a refresh token in an httpOnly, Secure, SameSite=Lax cookie. Rotate the refresh token on use.
- Seeded demo accounts (one per role) exist only when `DEMO_MODE=true`. The production flag disables them and the guest write endpoints.
- Login rate limit: 5 attempts per minute per IP and account, then a temporary lockout.

3.3 Threat model and controls

Threat	Control
Malicious or oversized upload	5 MB limit, allowlist JPEG/PNG/WebP, verify with Pillow, cap pixel count (decompression bomb guard), re-encode before inference, never execute or serve uploaded bytes
Location leakage in photos	Strip EXIF and GPS data on receipt
Prompt injection (chat text, text visible in an image sent to a vision model)	Treat all user content as data. Fixed system prompt, no tools or actions exposed to the model, output validated against a schema, advisor numbers checked against twin data
API key theft	Keys live only in server environment variables, never in the frontend bundle, `.env` is git-ignored, secret scanning enabled on the repo, rotate any key that ever touches a commit
Cost abuse and DoS	Per-IP and per-user rate limits, request size caps, LLM token caps, cached advice, global daily budget alarm
Broken access control (IDOR)	Ownership checks in the service layer, role dependency on every router, tests for cross-user access
XSS	Render model output as sanitized markdown, no `dangerouslySetInnerHTML`, strict Content-Security-Policy

Threat	Control
CSRF	SameSite cookies, bearer token on API calls, CORS allowlist of exact origins
SQL injection	ORM with parameterized queries only
Supply chain	Lockfiles, pinned versions, `pip-audit` and `npm audit` before submission, no unreviewed install scripts
Unsafe advice for hazardous items	Conservative defaults on uncertainty, on-screen note that guidance is informational and local rules apply

3.4 Data handling and privacy

- **Images:** processed in memory and discarded. If you add an optional 'save for review' feature, require explicit consent, store with a random filename, and delete after 7 days.
- **Minimization:** store class, subtype, confidence, timestamp, and chosen source node. No device identifiers, no precise location.
- **Personal data:** email and password hash only. Provide delete-my-data for residents.
- **Third parties:** if images or prompts go to an LLM provider, say so in the UI, send only what is needed, and prefer a provider setting that does not train on API data.
- **Principles followed:** consent, purpose limitation, and data minimization, in the spirit of India's DPDP Act 2023. This is a design intent, not a legal review.
- **Faces and people:** show a hint to frame only the item. Do not retain frames that include people.

3.5 Transport and platform hardening

- HTTPS everywhere (host-provided certificates), HSTS enabled.
- Response headers: Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, frame-ancestors none.
- Disable debug mode and verbose errors in production. Errors carry a `request_id`, never stack traces.
- Run containers as a non-root user with a minimal base image.
- Disable `/docs` and `/openapi.json` in production or put them behind admin.

3.6 Logging and audit

- Structured logs with `request_id`, user id (not email), route, status, latency.

- Never log images, tokens, passwords, or full prompts. Log a hash of the advisor input instead.
- Audit events: login, failed login, role change, demo reset, report export.

3.7 Pre-submission security checklist

- [ ] No secrets in git history (`git log -p` search for key prefixes)
- [ ] `.env.example` committed, `.env` ignored
- [ ] Demo accounts and guest limits correct for the deployed environment
- [ ] Rate limits verified with a quick loop test
- [ ] Upload test files: oversized, wrong type, corrupted, EXIF with GPS
- [ ] Prompt-injection test strings tried in chat
- [ ] `pip-audit` and `npm audit` reviewed
- [ ] CORS only allows the deployed frontend origin

4. Frontend Specification

4.1 Principles

- Mobile-first for the scan flow, desktop-first for the twin and what-if views.
- One clear result per scan: the class badge and the bin come first, details second.
- Never present simulated data as measured data. Label it.
- Fast: the scan page loads first, heavy views (twin graph, charts) are code-split.

4.2 Stack

React 18, TypeScript, Vite, React Router, TanStack Query (server state), Zustand (UI and scan session state), Tailwind CSS, React Flow (twin graph), Recharts (charts), Framer Motion (sparingly), a small accessible component set (Radix primitives or shadcn/ui).

4.3 Routes and screens

Route	Screen	Roles	Priority
`/`	Landing with 'Try demo' and a short value statement	All	P0
`/scan`	Camera or upload, result card, guidance, confirm-bin action	Guest+	P0
`/twin`	Live flow graph, node details, metric strip	Guest+	P0

Route	Screen	Roles	Priority
`/simulate`	What-if sliders, baseline vs scenario comparison	Guest+	P0
`/impact`	KPI cards and trend charts	Guest+	P0
`/advisor`	Structured advice cards and chat	Resident+	P0 (cards), P1 (chat)
`/history`	Past scans with filters	Resident+	P1
`/facility`	Bin fill levels, hotspot list, report export	Manager+	P1
`/login`	Sign in and demo accounts	Public	P1

4.4 Layout

A persistent shell: top bar (logo, role badge, demo-mode indicator, sign-in) and a left navigation on desktop that becomes a bottom tab bar on mobile (Scan, Twin, What-If, Impact, Advisor). A slim banner at the top reads 'Simulated campus data' whenever the twin is running on demo traffic.

4.5 Key components

Component	Purpose and behavior
`CameraCapture`	`getUserMedia` with `facingMode: environment`, capture to canvas, downscale to max 1024 px, JPEG 0.85, retake button, file-upload fallback when permission is denied
`ResultCard`	Class badge, subtype, confidence bar, top-3 list, bin and prep steps, 'I put it in…' confirm buttons, 'Not right?' report action
`UncertainState`	Shown below the confidence threshold: plain-language 'not sure', retake tip, cautious default when hazardous is plausible
`TwinGraph`	React Flow canvas with custom nodes (source, bin, collection, facility), edge thickness by flow, animated dashes for active flow, node click opens a detail drawer
`MetricStrip`	Diversion rate, contamination rate, CO2e avoided, pickup cost, with small delta arrows

Component	Purpose and behavior
`ScenarioPanel`	Sliders for compliance, awareness boost, pickup frequency, bins per stream, with reset and Run buttons
`CompareChart`	Baseline vs scenario bars plus delta KPI cards, colored by good or bad direction
`AdviceCards`	Ranked actions with impact and effort chips, expandable 'why' text, reuse ideas list
`AdvisorChat`	Streaming message list, suggested prompts, clear disclaimer
`StatusBanner`	Demo mode, offline, and degraded-AI messages

4.6 Visual system

- **Class colors (also used in the legend):** recyclable blue, reusable amber, biodegradable green, hazardous red, residual gray. Never rely on color alone: each badge also has an icon and a text label.
- **Typography:** one sans-serif family, large result text (at least 24 px class name on mobile), tabular numerals for metrics.
- **Spacing and shape:** 8 px grid, rounded cards, restrained shadows.
- **Themes:** light theme required; dark theme is a P2 task using CSS variables defined from day one.

4.7 State and data flow

- **TanStack Query** for `/twin`, `/scans`, `/advice`, with sensible stale times. Mutations for `/scan`, `/simulate`.
- **Zustand** for the current scan session (image, result), selected twin node, and scenario slider values.
- **`useTwinStream` hook** opens the SSE connection, patches the query cache on `scan.created` and `metrics.updated`, reconnects with backoff, and falls back to 3-second polling.
- **API client** (`api/client.ts`) attaches the token, maps the standard error shape to typed errors, and adds the `request_id` to bug reports.

4.8 Interaction details

- **Scan:** capture, show a loading skeleton, then the result within 3 seconds. Announce the result with `aria-live=polite`.
- **Confirm bin:** one tap on the bin the user actually used. This feeds real routing into the twin and is the source of the 'real vs simulated' split.
- **Twin:** newly scanned items animate along their path. A time slider replays the last N ticks.
- **What-if:** slider changes preview instantly with a debounced call (300 ms). 'Run full scenario' returns the complete comparison.
- **Advice:** a Generate button with a progress state. If the LLM is down, show cached advice with a 'cached' tag.
- **Report:** export from the facility page using the browser's print-to-PDF on a print stylesheet (the P1 backend report endpoint can replace this).

4.9 States and errors

Every screen defines loading (skeletons), empty (a short prompt plus an action), error (message plus retry and the `request_id`), and offline states. Camera permission denied shows the upload fallback. Rate-limited requests show a friendly wait message.

4.10 Accessibility

WCAG AA contrast, visible focus rings, full keyboard operation for sliders and dialogs, labels for all controls, alt text on the preview image, reduced-motion respected for animated edges, and touch targets of at least 44 px.

4.11 Performance budget

Initial JS under 250 KB gzipped on `/scan`, lazy-load `/twin` and `/simulate`, compress images client-side before upload, and memoize graph nodes to keep the canvas smooth with around 40 nodes.

4.12 Acceptance criteria

Screen	Done when
Scan	A photo from a phone returns a result card in under 3 s; denied camera falls back to upload; low-confidence path renders
Twin	Scanning an item updates a count on the graph within 1 s without a refresh

Screen	Done when
What-if	Moving compliance changes diversion and CO2e deltas; reset restores baseline
Impact	KPIs match the twin API values exactly
Advisor	Cards render from valid advice JSON; cached fallback works with the LLM disabled
All	No console errors on the demo path; works at 375 px and 1440 px widths

5. Feature Ticket List

Roles: **ML** (model and data), **BE** (backend, twin, advisor), **FE** (frontend). Estimates are in hours of focused work and are rough. Priorities: P0 must ship, P1 should ship, P2 only if time remains.

Capacity check. The P0 tickets add up to about 32 person-hours. A 3-person team has roughly 39 hours across both days, which is tight but workable. With 2 people (about 26 hours) use the cut list in 5.3 and plan some remote work in the evening, which the event allows. With 4 people there is slack for P1 work.

5.1 Tickets

ID	Ticket	Pri	Est	Role	Depends on	Done when
E0 Setup						
RC-001	Init monorepo, FastAPI and Vite skeletons, lint, `.env.example`	P0	0.75	BE, FE	none	Both apps start locally
RC-002	Seed files: class map, bin rules, emission factors (placeholders), twin topology	P0	0.5	BE	RC-001	Files load without errors
RC-003	Docker Compose and Makefile	P1	0.5	BE	RC-001	`docker compose up` works from clean
E1 Vision						
RC-010	Download dataset, map to 5 classes, make train/val/test split	P0	1.0	ML	RC-001	Class counts printed, split saved
RC-011	Fine-tune classifier and save weights	P0	1.5	ML	RC-010	Val accuracy at least 80% on first pass

ID	Ticket	Pri	Est	Role	Depends on	Done when
RC-012	Eval script: confusion matrix, per-class recall, own-photo set	P0	0.5	ML	RC-011	`metrics.json` and chart saved
RC-013	`POST /scan`: validate, preprocess, infer, top-3, guidance lookup	P0	1.0	BE	RC-011, RC-002	Returns the contract in 2.7 under 500 ms
RC-014	Low-confidence fallback and `unsure` path	P1	1.0	ML	RC-013	Blurry photo returns the unsure result
E2 Digital Twin						
RC-020	Twin graph schema, seed campus topology, `GET /twin`	P0	1.0	BE	RC-002	Graph and metrics return as JSON
RC-021	Scan-to-twin event ingestion	P0	0.75	BE	RC-013, RC-020	A scan changes node counts
RC-022	Simulation engine with seeded RNG (compliance, contamination, yield)	P0	2.0	BE	RC-020	Same seed gives the same output
RC-023	Metrics: diversion, contamination, CO2e, pickup cost	P0	1.0	BE	RC-022	Unit tests pass on hand-computed cases
RC-024	`POST /simulate` with baseline vs scenario deltas	P0	0.75	BE	RC-023	Deltas returned for all KPIs
RC-025	Demo traffic generator and SSE stream	P0	1.0	BE	RC-021	Events stream to a curl client
E3 Advisor						
RC-030	LLM client: provider config, failover, timeout, cache	P0	1.0	BE	RC-001	Fallback triggers when primary is forced to fail
RC-031	Advice prompt, Pydantic schema, number validator, template fallback	P0	1.5	BE	RC-030, RC-023	Invented figures are rejected in a test
RC-032	`POST /advice` with cached sample for guests	P0	0.5	BE	RC-031	Works with the LLM disabled
RC-033	`POST /chat` with streaming	P1	1.0	BE	RC-030	Tokens stream to the client
RC-034	Report endpoint (markdown to PDF)	P2	1.0	BE	RC-031	Downloadable report
E4 Frontend						

ID	Ticket	Pri	Est	Role	Depends on	Done when
RC-040	App shell, routing, tokens, API client, status banner	P0	1.0	FE	RC-001	All routes render placeholders
RC-041	`CameraCapture` with upload fallback and client downscale	P0	1.25	FE	RC-040	Works on a phone browser
RC-042	`ResultCard`, `UncertainState`, confirm-bin action	P0	1.0	FE	RC-041, RC-013	Result shows within 3 s
RC-043	`TwinGraph`, `MetricStrip`, `useTwinStream`	P0	2.5	FE	RC-020, RC-025, RC-040	Count updates live after a scan
RC-044	Simulate page: `ScenarioPanel`, `CompareChart`	P0	2.0	FE	RC-024	Slider change shows deltas
RC-045	Impact page	P0	1.0	FE	RC-023	KPIs match the API
RC-046	`AdviceCards`	P0	1.25	FE	RC-032	Cards render from sample JSON
RC-047	`AdvisorChat`	P1	1.0	FE	RC-033	Streaming reply visible
RC-048	History page	P1	0.75	FE	RC-013	Lists past scans
RC-049	Facility page (bin fill, hotspots)	P1	1.25	FE	RC-021	Hotspot list renders
RC-050	Loading, empty, error states, accessibility and mobile pass	P0	1.25	FE	RC-042, RC-043, RC-044	Passes the checks in 4.10
E5 Auth & Security						
RC-060	JWT auth, roles, demo accounts	P1	1.5	BE	RC-001	Role matrix enforced in tests
RC-061	Upload hardening, rate limits, CORS, security headers	P0	1.0	BE	RC-013	Checklist items in 3.7 pass
RC-062	Prompt-injection guards and secrets check	P0	0.5	BE	RC-030	Injection test strings do not change behavior
E6 Deploy & Submit						
RC-070	Backend deploy (Docker) and env wiring	P0	1.0	BE	RC-013	Public `/health` returns OK
RC-071	Frontend deploy	P0	0.5	FE	RC-040	Public URL loads and calls the API
RC-072	Smoke test script, warm-up routine, backup video, local fallback	P0	0.75	All	RC-070, RC-071	`smoke_test.sh` exits 0

ID	Ticket	Pri	Est	Role	Depends on	Done when
RC-073	README, repo hygiene, Google Drive folder with project files	P0	0.5	FE	none	A stranger can run the project
RC-074	Slide deck and 3 demo rehearsals	P0	1.5	All	RC-072	3 clean runs under 3 minutes
RC-075	Submit the form (PPT, GitHub, deployed link, Drive)	P0	0.25	All	RC-074	Confirmation received

5.2 Schedule (indicative; adjust to the real break times)

Block	Focus	Milestone
Day 1, 10:00 to 10:45	Kickoff and briefing, repo init (RC-001, RC-002)	Skeletons running
Day 1, 10:45 to 12:30	ML: dataset and training start (RC-010, RC-011). BE: twin schema (RC-020). FE: shell (RC-040)	Training running while others build
Day 1, afternoon to 4:30	ML eval, `/scan`, ingestion, engine, camera and result UI (RC-012, 013, 021, 022, 041, 042)	**M1: live scan shows a result in the UI**
Day 2, 10:00 to 1:00	Metrics, simulate, advisor, twin UI, simulate UI, impact, advice cards	**M2: scan to twin to what-if to advice, end to end**
Day 2, 1:00	**P0 feature freeze**	Bug fixes only
Day 2, 1:00 to 2:30	Deploy, hardening, accessibility pass, smoke test	Deployed and warm
Day 2, 2:30 to 3:45	Deck, 3 rehearsals, backup video	Demo ready
Day 2, by 4:00	Submit (hard deadline is 4:30)	**Submitted with buffer**

5.3 Cut list for a 2-person team

In this order, drop or simplify: replace SSE with 3-second polling (RC-025, saves about 0.5 h); render a static twin graph without edge animation (RC-043, saves about 1 h); fold the Impact page into the twin metric strip (RC-045, saves about 1 h); use the template advice only plus a cached sample (RC-031, saves about 0.75 h); skip RC-014, RC-033, RC-047 to RC-049, RC-060.

5.4 Definition of done

A ticket is done when its 'Done when' check passes, the code is merged to `main`, it has no console or server errors on the demo path, and any new env variable is in `.env.example`.

6. Folder & File Structure

6.1 Repository tree

```
recircle/
├── README.md                    # what it is, quick start, architecture
image, team
├── docker-compose.yml           # api + web for local and backup demo
├── Makefile                     # make dev, make test, make train, make
smoke
├── .env.example                 # every variable, no secrets
├── .gitignore                   # .env, weights/, node_modules,
__pycache__
├── docs/
│   ├── prd.md
│   ├── trd.md
│   ├── security.md
│   ├── frontend-spec.md
│   ├── tickets.md
│   ├── architecture.png
│   └── demo-script.md
├── ml/
│   ├── README.md                # dataset source, licence, how to
reproduce
│   ├── configs/
│   │   └── classes.yaml         # source label to output class and subtype
│   ├── scripts/
│   │   ├── download_data.py
│   │   ├── prepare_dataset.py   # mapping and train/val/test split
│   │   ├── train.py
│   │   ├── evaluate.py          # confusion matrix, per-class recall
│   │   └── export.py            # writes weights for the backend
│   ├── own_photos/              # small realism test set (committed or on
Drive)
│   ├── reports/                 # metrics.json, confusion_matrix.png
│   └── weights/                 # git-ignored; downloaded or copied at
deploy
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
```

```
|   ├── app/
|   |   ├── main.py                # app factory, middleware, router mount
|   |   ├── core/
|   |   |   ├── config.py          # settings from env
|   |   |   ├── security.py        # hashing, JWT, role dependencies
|   |   |   ├── rate_limit.py
|   |   |   └── logging.py         # structured logs, request_id
|   |   ├── api/v1/
|   |   |   ├── router.py
|   |   |   ├── auth.py
|   |   |   ├── scan.py
|   |   |   ├── history.py
|   |   |   ├── twin.py            # GET /twin and /twin/stream
|   |   |   ├── simulate.py
|   |   |   ├── advice.py
|   |   |   ├── chat.py
|   |   |   ├── reports.py
|   |   |   ├── admin.py           # demo reset and traffic
|   |   |   └── health.py
|   |   ├── models/                # SQLAlchemy tables
|   |   |   ├── user.py
|   |   |   ├── scan.py
|   |   |   ├── twin.py
|   |   |   └── advice.py
|   |   ├── schemas/               # Pydantic request and response models
|   |   ├── services/
|   |   |   ├── vision/
|   |   |   |   ├── classifier.py  # model load and predict
|   |   |   |   ├── preprocess.py  # validate, strip EXIF, resize
|   |   |   |   ├── fallback.py    # low-confidence path
|   |   |   |   └── guidance.py    # class to bin and steps
|   |   |   ├── twin/
|   |   |   |   ├── graph.py       # nodes, edges, state
|   |   |   |   ├── engine.py      # tick simulation
|   |   |   |   ├── metrics.py
|   |   |   |   ├── events.py      # scan ingestion, SSE bus
|   |   |   |   └── generator.py   # demo traffic
|   |   |   ├── advisor/
|   |   |   |   ├── llm_client.py  # providers, failover, cache
|   |   |   |   ├── prompts.py
|   |   |   |   ├── advisor.py     # build input, call, validate
|   |   |   |   └── guard.py       # injection and number checks
|   |   |   └── reports/
|   |   ├── data/
|   |   |   ├── classes.json
|   |   |   ├── bin_rules.json
```

```
|   |   |   ├── emission_factors.json
|   |   |   ├── twin_seed.json
|   |   |   ├── sample_advice.json # cached advice for guests and LLM outage
|   |   |   └── knowledge/         # short disposal and reuse notes
(markdown)
|   |   └── db/
|   |       ├── session.py
|   |       └── seed.py            # demo users, topology, history
|   └── tests/
|       ├── test_twin_engine.py    # determinism and metric cases
|       ├── test_scan_api.py       # upload validation, contract
|       ├── test_advisor_guard.py  # invented numbers rejected
|       └── test_access.py         # role matrix and IDOR
├── frontend/
|   ├── package.json
|   ├── vite.config.ts
|   ├── tailwind.config.ts
|   ├── tsconfig.json
|   ├── index.html
|   ├── public/
|   └── src/
|       ├── main.tsx
|       ├── App.tsx
|       ├── routes.tsx
|       ├── api/
|       |   ├── client.ts          # fetch wrapper, auth, typed errors
|       |   ├── scan.ts
|       |   ├── twin.ts
|       |   ├── simulate.ts
|       |   ├── advice.ts
|       |   └── auth.ts
|       ├── pages/
|       |   ├── Landing.tsx
|       |   ├── Scan.tsx
|       |   ├── Twin.tsx
|       |   ├── Simulate.tsx
|       |   ├── Impact.tsx
|       |   ├── Advisor.tsx
|       |   ├── History.tsx
|       |   ├── Facility.tsx
|       |   └── Login.tsx
|       ├── components/
|       |   ├── ui/                # buttons, cards, badges, sliders, drawer
|       |   ├── layout/            # Shell, NavBar, TabBar, StatusBanner
|       |   ├── scan/              # CameraCapture, ResultCard,
UncertainState
```

```
|       |   ├── twin/              # TwinGraph, nodes, MetricStrip,
NodeDrawer
|       |   ├── simulate/          # ScenarioPanel, CompareChart
|       |   └── advisor/           # AdviceCards, AdvisorChat
|       ├── hooks/                 # useTwinStream, useCamera, useDebounce
|       ├── stores/                # scanSession.ts, scenario.ts, ui.ts
|       ├── lib/                   # image.ts (downscale), format.ts,
classMeta.ts
|       ├── types/                 # API types shared across the app
|       └── styles/                # tokens.css, globals.css
└── scripts/
    ├── smoke_test.sh             # hits health, scan sample, twin,
simulate, advice
    ├── warmup.sh                 # pings the deployed API before the demo
    └── seed_demo.sh              # resets demo data
```

6.2 Environment variables (all in `.env.example`)

Variable	Used by	Purpose
`APP_ENV`	API	`dev` or `prod`
`DEMO_MODE`	API	Enables demo accounts and guest endpoints
`DATABASE_URL`	API	SQLite path or Postgres URL
`JWT_SECRET`, `JWT_ACCESS_MINUTES`	API	Token signing and lifetime
`CORS_ORIGINS`	API	Exact allowed frontend origins
`MODEL_PATH`, `CONFIDENCE_THRESHOLD`	API	Weights file and unsure cut-off
`MAX_UPLOAD_MB`, `RATE_LIMIT_PER_MIN`	API	Upload size and request limits
`LLM_PRIMARY_PROVIDER`, `LLM_PRIMARY_MODEL`, `LLM_PRIMARY_API_KEY`	API	Main advisor model
`LLM_FALLBACK_PROVIDER`, `LLM_FALLBACK_MODEL`, `LLM_FALLBACK_API_KEY`	API	Backup model
`LLM_TIMEOUT_SECONDS`	API	Per-call timeout
`VITE_API_BASE_URL`	Web	Backend base URL (public value, no secrets)

6.3 Conventions

- **Branches:** `main` is always deployable. Work on `feat/RC-0xx-short-name`, merge by small pull request or fast-forward after a quick review.
- **Commits:** start with the ticket id, for example `RC-022 add tick loop and seeded RNG`.
- **Ports:** API 8000, web 5173.
- **Naming:** Python in snake_case, React components in PascalCase, one component per file, API types mirrored in `frontend/src/types`.
- **Contract first:** agree the JSON shapes in section 2.7 before coding. Backend and frontend then build in parallel against a mock if needed.

- **README must include:** one-paragraph pitch, architecture image, quick start (`make dev`), env setup, dataset and licence note, team roles, and the demo credentials for judges.
- **Submission bundle:** PPT, GitHub link, deployed link, and a Drive folder with the project files, submitted through the form before 4:30 PM on 16 October.

7. Additions

Two additions to the pack. Sections 1 to 6 are unchanged. Both items are P2 and sit on top of the existing twin, so they can be dropped without affecting any P0 or P1 work.

7.1 Sankey material-flow view

What it is. A second way to look at the Digital Twin: one diagram that shows items moving from sources to bins to facilities, with link width proportional to the number of items. The React Flow graph in 4.5 stays the default. The Sankey is a toggle on the same page (*Graph* | *Flow*) and answers a different question: not how the system is wired, but where the material actually ends up.

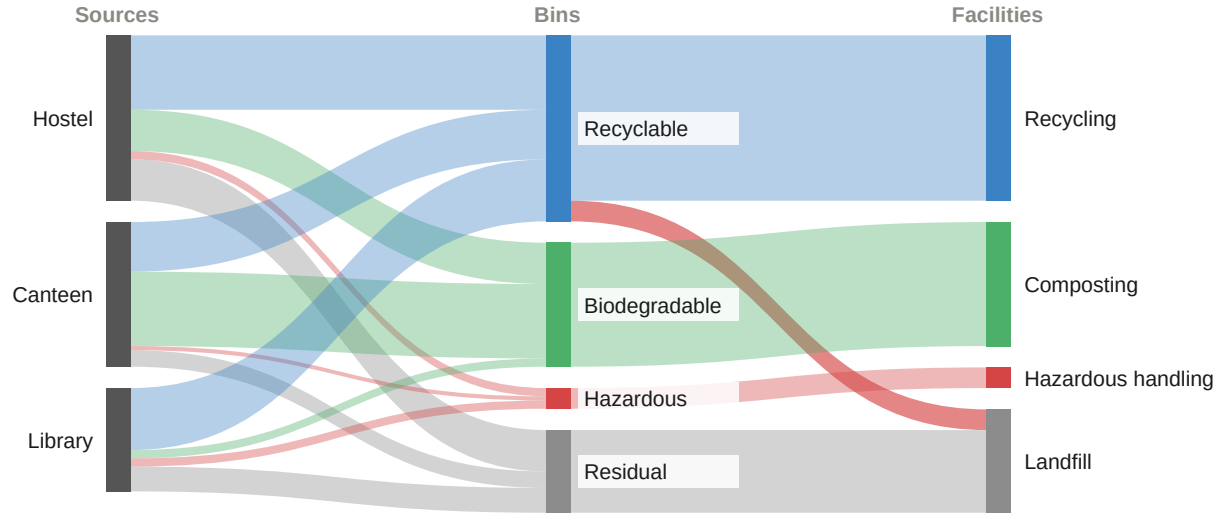


Figure 7.1. Illustrative layout only, with made-up numbers. The red link from Recyclable to Landfill is contamination: recyclable-bin items that were downgraded and landfilled.

Why it helps the pitch. Judges see the contamination loss as a physical leak in one diagram, and the what-if demo in 1.11 becomes visual: raise compliance and the red link visibly shrinks.

Data. No new endpoint. The client derives links from the existing `GET /twin` payload (`twin_edges.weight` by `from_id` and `to_id`) and the contaminated count per bin. Layers: Source, Bin, Collection (merged into the bin-to-facility link), Facility, Outcome.

Library. Use the Sankey component that ships with Recharts, which is already in the stack (4.2), so there is no new dependency. If it proves too limiting, `d3-sankey` is the fallback. Lazy-load it with the twin route so the 250 KB budget in 4.11 holds.

Behavior and acceptance criteria

Aspect	Spec
Placement	Toggle on `/twin` next to the graph view. Optional side-by-side baseline vs scenario on `/simulate` (stretch).
Colors	Link color follows the class colors in 4.6 (recyclable blue, biodegradable green, hazardous red, residual gray). Contaminated links use red with a hatch or dashed edge, and a legend entry, so color is never the only cue.
Real vs simulated	The 'Simulated campus data' banner in 4.4 stays visible. Links from confirmed user scans are tagged *real* in the tooltip. Everything else is tagged *simulated*.
Interaction	Hover shows items, estimated mass, and share of total. Click opens the existing NodeDrawer.
Live update	A scan changes link widths within 1 s, using the same SSE patch as the graph (4.7).
Mobile	At 375 px the diagram scrolls horizontally inside its card. The page itself must not scroll sideways.
Accessibility	A visually hidden table lists every link (source, target, items) as the screen-reader alternative.
Done when	The diagram renders from live `/twin` data, link widths match the API counts, and the contamination link changes when the compliance slider is moved and the scenario is run.

Additional tickets

ID	Ticket	Pri	Est	Role	Depends on	Done when
RC-080	Flow data adapter: map /twin nodes and edges to Sankey links, including the contaminated link	P2	0.5	FE	RC-020, RC-043	Unit test: link values equal edge weights
RC-081	FlowSankey component, Graph/Flow toggle, legend, tooltip, hidden table	P2	1.5	FE	RC-080	Scan updates a link within 1 s
RC-082	Baseline vs scenario Sankey on /simulate (stretch)	P2	1.0	FE	RC-044, RC-081	Both diagrams render from one /simulate response

Place new files at `frontend/src/components/twin/FlowSankey.tsx` and `frontend/src/lib/sankey.ts`. Add RC-080 and RC-081 to the cut list in 5.3 as the first items to drop.

7.2 Future work

A roadmap for the closing slide and the README. None of this is built during the event, which keeps the build inside the rule that all code is written on site and inside the non-goals in 1.4. Each item is chosen because the current design already leaves a place for it.

Idea	What it adds	Where it plugs in	First step
Resin-code and symbol reading	Finer plastic subtypes (the number inside the recycling triangle) and better guidance on which plastics a local facility accepts.	Extra output on `POST /scan` feeding the subtype field and `bin_rules.json`.	Collect close-up photos of resin codes. Try a small detector or OCR pass on the cropped region.

Idea	What it adds	Where it plugs in	First step
Multi-item detection	Several items in one frame, each routed on its own. Faster scanning for a whole bag of waste.	Replace the classifier in `classifier.py` with a detector. One scan event per box.	Fine-tune a small detector on a few hundred labelled multi-item photos.
Smart-bin prototype	A camera and a servo lid that routes the item automatically. This replaces the manual 'I put it in...' confirmation with real routing.	A device posts to `POST /scan` with a bin id. The twin treats it as a confirmed, real event.	Prototype with a Raspberry Pi, a camera module and one servo on a single bin.
Campus pilot	Replace simulated sources with real bin-level counts from one hostel or canteen.	Seed topology in `twin_seed.json` becomes configurable by a Manager (3.1).	Agree one location and one data-sharing arrangement with the facilities office.

Suggested closing-slide line. Today ReCircle scans one item at a time and simulates the campus. Next: read resin codes, detect many items at once, put the camera in the bin, and replace simulated sources with a real pilot.